

Informe Laboratorio 1

Sección 1

Cinthya Alejandra Fuentealba Bravo
cinthya.fuentealba@mail.udp.cl

04 de Septiembre de 2026

Índice

1. Descripción	2
2. Actividades	2
2.1. Algoritmo de cifrado	2
2.2. Modo stealth	2
2.3. MitM	3
3. Desarrollo de Actividades	4
3.1. Actividad 1	4
3.2. Actividad 2	5
3.3. Actividad 3	12
4. Conclusiones y comentarios	16
5. Uso de IA	17

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a IA Generativa y validar si el código resultante cumple con lo requerido.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en python3 utilizando IA Generativa, que permita cifrar texto utilizando el algoritmo César. Como parámetros de su programa deberá ingresar el string a cifrar y luego el desplazamiento.

```

$ sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb

```

2.2. Modo stealth

1. Generar un programa, en python3 utilizando IA Generativa, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

```

$ sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.

```

El último carácter del mensaje se transmite como una b.

Data (48 bytes)	
Data: 6260090000000000101112131415161718191a1b1c1d1e1f20212223242526272	
[Length: 48]	
0000	ff ff ff ff ff ff 00 00 00 00 00 00 08 00 45 00E.
0010	00 54 00 01 00 00 40 01 76 9b 7f 00 00 01 7f 06 .T....@. v.....
0020	06 06 08 00 56 83 00 01 00 21 64 22 13 05 00 00V... !d"....
0030	00 00 62 60 09 00 00 00 00 00 10 11 12 13 14 15 ..b`.....
0040	16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 !"#\$\$%
0050	26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 &'()*+,-./012345
0060	36 37 67

2.3. MitM

1. Generar un programa, en python3 utilizando IA Generativa, que permita obtener el mensaje transmitido en el paso2. Como no se sabe cual es el desplazamiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```

$ sudo python3 readv2.py cesar.pcapng
0      larycxpajorj h bnpdarmjm nw anmnb
1      kzqxbwozinqi g amoczqlil mv zmlma
2      jypwavnyhmpf f zlnbypkhh lu yklkz
3      ixovzumxglog e ykmaxojgj kt xkjky
4      hwnuytlwfknd d xjlzwnifi js wjiyx
5      gvmtxskvejme c wikyvmeheh ir vihiw
6      fulswrjudild b vhjxulgdg hq uhghv
7      etkrvqitchkc a ugiwtkfcf gp tgfgu
8      dsjquphsbgjb z tfhvsjebe fo sfeft
9      criptografia y seguridad en redes
10     bqhosnfqzehz x rdftqhczc dm qdcdr
11     apgnrmepdygy w qcespgbyb cl pcbcq
12     zofmqldoxcfx v pbdrofaxa bk obabp
13     ynelpkcnwbew u oacqnezwz aj nazao
14     xmdkojbmadv t nzbpmdivy zi mzyzn
15     wlcjnia luzcu s myaolcxux yh lyxym
16     vkbimhzktybt r lxznkbwtw xg kxwxl
17     ujahlgysxas q kwymjavsv wf jwvwk
18     tizgkfxirwzr p jvxlizuru ve ivuvj
19     shyfjewhqvyq o iuwkhytqt ud hutui
20     rgxeidvgpuxp n htvjgxspz tc gtsth
21     qfwdhcufotwo m gsuifwrwr sb fsrsg
22     pevcbtensvn l frthevqng ra erqrf
23     odubfasdmrum k eqsgdupmp qz dqpqe
24     nctaezrclqtl j dprfctolo py cpopd
25     mbszdyqbksk i coqebnkn ox bonoc

```

Finalmente, deberá indicar por qué eligió esa IA Generativa en relación a otras del mercado. Comente si tuvo algún problema al momento de generar los códigos.

3. Desarrollo de Actividades

En este primer laboratorio, el objetivo principal es analizar la posibilidad de transmitir información mediante tráfico de red aparentemente legítimo y demostrar que la información puede ser recuperada al tener en conocimiento el método utilizado para su cifrado y transmisión.

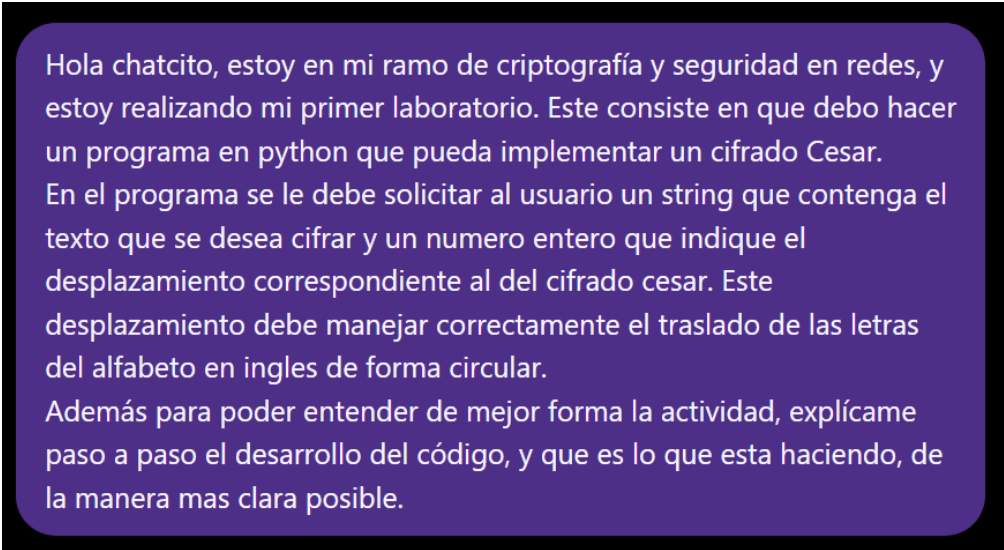
Para la realización de este laboratorio, se desarrollan tres actividades principales, en las cuales se utilizaron las siguientes herramientas tecnológicas: Linux, Python 3, herramienta de IA y Wireshark. La herramienta de IA utilizada durante este laboratorio es **ChatGPT**.

A continuación se presentan las tres actividades realizadas:

3.1. Actividad 1

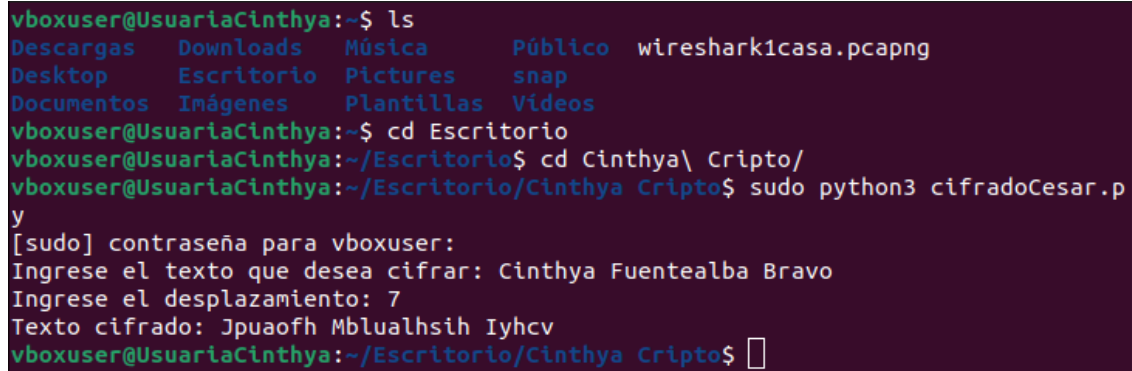
La primera actividad realizada en este laboratorio es la implementación de un programa en Python 3, capaz de cifrar un mensaje utilizando el algoritmo de cifrado César. Este tipo de cifrado permite desplazar cada letra de un mensaje una determinada cantidad de posiciones dentro del alfabeto.

El programa debe recibir como parámetros un **string** y un **entero**. El string corresponde al texto a cifrar, mientras que el número entero recibido como parámetro corresponde a la cantidad de desplazamiento utilizado para el cifrado.



Hola chatcito, estoy en mi ramo de criptografía y seguridad en redes, y estoy realizando mi primer laboratorio. Este consiste en que debo hacer un programa en python que pueda implementar un cifrado Cesar. En el programa se le debe solicitar al usuario un string que contenga el texto que se desea cifrar y un numero entero que indique el desplazamiento correspondiente al del cifrado cesar. Este desplazamiento debe manejar correctamente el traslado de las letras del alfabeto en ingles de forma circular. Además para poder entender de mejor forma la actividad, explícame paso a paso el desarrollo del código, y que es lo que esta haciendo, de la manera mas clara posible.

Figura 1: Prompt utilizado en la herramienta de IA (ChatGPT) para la generación del código en Python 3, para la actividad número 1 y su especificación de la utilización del alfabeto en inglés.



```
vboxuser@UsuaríaCinthya:~$ ls
Descargas  Downloads  Música     Público   wireshark1casa.pcapng
Desktop    Escritorio  Pictures    snap
Documentos Imágenes   Plantillas Videos
vboxuser@UsuaríaCinthya:~$ cd Escritorio
vboxuser@UsuaríaCinthya:~/Escritorio$ cd Cinthya\ Cripto/
vboxuser@UsuaríaCinthya:~/Escritorio/Cinthya Cripto$ sudo python3 cifradoCesar.py
[sudo] contraseña para vboxuser:
Ingrese el texto que desea cifrar: Cinthya Fuentealba Bravo
Ingrese el desplazamiento: 7
Texto cifrado: Jpuaofh Mblualhsih Iyhcv
vboxuser@UsuaríaCinthya:~/Escritorio/Cinthya Cripto$
```

Figura 2: Captura de terminal en Linux ejecutando el programa en Python 3 para la primera actividad, llamado `cifradoCesar.py`.

Como se puede apreciar en la Figura 2, el programa en Python 3 nos pide ingresar el texto que se desea cifrar, en este caso el texto escogido es “Cinthya Fuentealba Bravo”, una vez ingresado el texto, se solicita en pantalla ingresar un entero correspondiente al desplazamiento en el cifrado, en este caso se escogió el número 7. Posteriormente, en el terminal muestra por pantalla el resultado con éxito, mostrando como queda el texto cifrado e ilegible.

3.2. Actividad 2

La segunda actividad de este laboratorio consistió en desarrollar un programa en Python 3 capaz de transmitir el mensaje cifrado obtenido en la actividad anterior mediante paquetes ICMP Echo Request.

La transmisión de este mensaje cifrado no solo debe cumplir con ser enviado mediante paquetes ICMP Request, sino que también debe cumplir la condición de enviar un carácter por paquete, es decir, los paquetes enviados deben ser carácter por carácter de manera individual con respecto al texto cifrado. El envío de estos paquetes se realizan incorporando cada carácter en el campo Data del paquete ICMP.

En primer lugar se debe realizar una captura en Wireshark, permitiendo analizar un tráfico generado mediante un `ping 8.8.8.8`, de esta manera se logra identificar la anatomía de un paquete ICMP y utilizarla como referencia para comparar posteriormente el tráfico generado.

```
vboxuser@UsuaríaCinthya:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=9.30 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=9.87 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=8.80 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=11.8 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=64 time=11.5 ms
^C
--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4710ms
rtt min/avg/max/mdev = 8.795/10.251/11.755/1.193 ms
vboxuser@UsuaríaCinthya:~$
```

Figura 3: Captura de la terminal de Linux realizando un ping a 8.8.8.8, transmitiéndose un total de 5 paquetes. En Wireshark se observa que los paquetes corresponden al protocolo ICMP y son solicitudes Echo Request, identificadas mediante Type 8 y Code 0.

(icmp) && (ip.dst == 8.8.8.8)						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
3	1.006330290	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
5	2.414143771	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
7	3.684949740	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
9	4.709677263	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request

▶ Frame 5: 100 bytes on wire (800 bits), 100 bytes captured (800 bits) on interface ar ▶ Linux cooked capture v1 ▶ Internet Protocol Version 4, Src: 10.0.2.15, Dst: 8.8.8.8 ▼ Internet Control Message Protocol						
Type: 8 (Echo (ping) request) Code: 0 Checksum: 0x4238 [correct] [Checksum Status: Good] Identifier (BE): 2 (0x0002) Identifier (LE): 512 (0x0200) Sequence Number (BE): 3 (0x0003) Sequence Number (LE): 768 (0x0300) [Response frame: 6] Timestamp from icmp data: Sep 4, 2026 01:49:55.000000000 -04 [Timestamp from icmp data (relative): 0.862562204 seconds] ▼ Data (48 bytes) Data: 4c290d0000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b...						
0000	00 04 00 01 00 06 08 00	27 03 2d c8 00 00 08 00	'.....			
0010	45 00 00 54 15 62 40 00	40 01 09 29 0a 00 02 0f	E..T.b@. @..)....			
0020	08 08 08 08 08 00 42 38	00 02 00 03 03 5c 9a 6a	B8\.j			
0030	00 00 00 00 4c 29 0d 00	00 00 00 00 10 11 12 13	L)..			
0040	14 15 16 17 18 19 1a 1b	1c 1d 1e 1f 20 21 22 23	 !"#			
0050	24 25 26 27 28 29 2a 2b	2c 2d 2e 2f 30 31 32 33	\$%&'()*+,-./0123			
0060	34 35 36 37		4567			

Figura 4: Captura en Wireshark, filtrando por paquetes de tipo ICMP y por su dirección IP, mostrando los paquetes correspondientes al ping realizado (captura original). Evidenciando que los paquetes de este tipo terminan peculiarmente todos en la data con los mismos caracteres, los cuales son: !"#\$%&'()*+,-./01234567

En la Figura 4 se puede observar en detalle un paquete ICMP generado mediante el comando ping. Se identifican campos como el Identifier y el Sequence Number, utilizados para relacionar y ordenar las solicitudes ICMP.

Además, el campo Data contiene información temporal y una secuencia característica de bytes utilizada por la implementación del comando ping. Esta estructura fue utilizada como referencia para comparar posteriormente los paquetes generados por el programa desarrollado.

El paquete corresponde a una solicitud Echo Request, identificada por el Type 8 y Code 0. Además, se observa un identificador ICMP con valor 0x0002 y un número de secuencia 0x0003, campos que permiten identificar y mantener el orden de las solicitudes enviadas.

También se observa que el paquete contiene un campo de datos de 48 bytes, además de información temporal incorporada por la herramienta ping. La secuencia hexadecimal presente en el payload mantiene una estructura característica, la cual fue utilizada como referencia para analizar la construcción de los paquetes generados posteriormente por el programa.

Característica	Ping Real	Paquete Generado
Protocolo	ICMP	ICMP
Tipo	Echo Request	Echo Request
Destino	8.8.8.8	8.8.8.8
Información transmitida	Datos del ping	Caracteres del mensaje cifrado
Cantidad de Paquetes	Según ejecución	Un paquete por carácter
Payload Payload	Estructura original de 48 bytes	Payload modificado para incorporar un carácter
Identifier y Sequence Number	Generados por ping	Diferentes a los observados en el ping original

Tabla 1: Comparación general entre un ping real y el tráfico generado por el programa.

La Tabla 1 permite comparar el tráfico generado mediante un ping convencional con el producido por el programa desarrollado. En ambos casos se utilizan paquetes ICMP de tipo Echo Request dirigidos a la misma dirección IP de destino.

La principal diferencia se encuentra en el campo *Data*. Mientras que el comando ping utiliza un payload generado por la propia herramienta, el programa desarrollado incorpora en este campo los caracteres correspondientes al mensaje cifrado. Para ello, se transmite un carácter por cada paquete ICMP, distribuyendo el mensaje completo entre múltiples solicitudes y permitiendo su posterior reconstrucción según el orden de recepción.

A partir del análisis realizado en Wireshark, se utilizaron como referencia las características del tráfico generado por un ping real. Posteriormente, se compararon estos paquetes con los generados por el programa desarrollado, identificando tanto las similitudes como las diferencias entre ambos.

Las capturas obtenidas permiten comprobar que ambos tipos de tráfico corresponden a

solicitudes ICMP Echo Request (Type 8, Code 0). Sin embargo, existen diferencias en la estructura del campo *Data*. El paquete generado mediante el programa contiene 56 bytes de datos, mientras que el paquete original analizado contiene 48 bytes e incluye información temporal generada por la herramienta **ping**. Por lo tanto, aunque se logró transmitir correctamente un carácter del mensaje cifrado en cada paquete, la estructura del payload no replica completamente la utilizada por un ping convencional.

También se observaron diferencias en los valores del *Identifier* y del *Sequence Number*. No obstante, el orden de los paquetes permite reconstruir correctamente el mensaje transmitido carácter por carácter.

Luego del análisis del tráfico ICMP, se solicitó a ChatGPT la generación de un programa en Python 3 que permitiera realizar la transmisión del mensaje cifrado mediante paquetes ICMP. El prompt realizado se presenta en la Figura 5.

Finalmente, las capturas del primer y último paquete permiten evidenciar la transmisión individual de los caracteres. En el último paquete, el payload comienza con el valor hexadecimal 76, correspondiente al carácter **v**, lo que confirma que el carácter final del mensaje cifrado fue incorporado correctamente en el campo de datos del paquete ICMP.

En esta segunda parte de mi laboratorio de criptografía y seguridad en redes lo que necesito hacer es crear en python, un nuevo código que haga lo siguiente:

El programa debe recibir como entrada un string que corresponde al resultado obtenido previamente mediante un programa de cifrado César.

Entonces el programa debe hacer lo siguiente:

El programa debe enviar ese string mediante paquetes ICMP echo request, debe solicitar al usuario mediante input el string cifrado obtenido del programa de cifrado cesar, luego se debe solicitar mediante input la dirección IP de destino, despues debe recorrer el string carácter por carácter.

Luego crear un paquete ICMP echo request independiente para cada uno de los caracteres. El carácter actual debe ser colocado exactamente como único contenido del campo data del paquete ICMP. No se deben agregar caracteres adicionales, padding, saltos de línea, espacios ni información adicional al campo data. Debe convertir cada carácter a bytes correctamente antes de incorporarlo al campo data y mantener exactamente el contenido y el orden del string original, incluyendo letras mayúsculas, minúsculas, números, espacios y signos de puntuación.

Ademas, se debe utilizar scapy para construir y enviar los paquetes.

Lo mas importante es que el paquete debe ser un ICMP echo request, similar al tráfico generado por un ping normal. Después de enviar cada paquete, mostrar en la terminal qué carácter fue enviado y su número de paquete. Por ejemplo: paquete 1 enviado: y la letra enviada

El programa no debe modificar el string recibido ni volver a cifrarlo.

El código debe ser compatible con Python 3 y Scapy.

Y obviamente debes explicarme el código línea por línea y especialmente explica.

También incluye instrucciones para ejecutarlo desde una terminal Linux.

Incluye una prueba de verificación para comprobar en Wireshark que cada paquete contiene exactamente un carácter en ICMP data y que la secuencia de caracteres coincide exactamente con el string enviado.

Mostrar menos ^

Figura 5: Prompt utilizado en la herramienta de IA (ChatGPT) para la generación del código en Python 3, para la actividad número 2, con sus respectivas especificaciones.

El programa solicita como entrada el texto cifrado obtenido en la actividad anterior y la

dirección IP de destino. Posteriormente, recorre el mensaje carácter por carácter y genera un paquete ICMP Echo Request para cada uno de ellos.

Cada carácter es incorporado dentro del campo Data del paquete, permitiendo que el mensaje completo sea transmitido mediante múltiples paquetes ICMP en lugar de aparecer completo dentro de una única transmisión.

Al ejecutar el programa de Python 3 en esta segunda actividad, se muestra lo siguiente en la terminal:

```
vboxuser@UsuaríaCinthy:~/Escritorio/Cinthy Cripto$ sudo python3 actividad2.py
Ingrese el texto cifrado: Jpuaofh Mblualhsih Iyhcv
Ingrese la IP de destino: 8.8.8.8
Paquete 1 enviado: 'J' | IP ID=1 | ICMP ID=1 | SEQ=1
Paquete 2 enviado: 'p' | IP ID=2 | ICMP ID=1 | SEQ=2
Paquete 3 enviado: 'u' | IP ID=3 | ICMP ID=1 | SEQ=3
Paquete 4 enviado: 'a' | IP ID=4 | ICMP ID=1 | SEQ=4
Paquete 5 enviado: 'o' | IP ID=5 | ICMP ID=1 | SEQ=5
Paquete 6 enviado: 'f' | IP ID=6 | ICMP ID=1 | SEQ=6
Paquete 7 enviado: 'h' | IP ID=7 | ICMP ID=1 | SEQ=7
Paquete 8 enviado: ' ' | IP ID=8 | ICMP ID=1 | SEQ=8
Paquete 9 enviado: 'M' | IP ID=9 | ICMP ID=1 | SEQ=9
Paquete 10 enviado: 'b' | IP ID=10 | ICMP ID=1 | SEQ=10
Paquete 11 enviado: 'l' | IP ID=11 | ICMP ID=1 | SEQ=11
Paquete 12 enviado: 'u' | IP ID=12 | ICMP ID=1 | SEQ=12
Paquete 13 enviado: 'a' | IP ID=13 | ICMP ID=1 | SEQ=13
Paquete 14 enviado: 'l' | IP ID=14 | ICMP ID=1 | SEQ=14
Paquete 15 enviado: 'h' | IP ID=15 | ICMP ID=1 | SEQ=15
Paquete 16 enviado: 's' | IP ID=16 | ICMP ID=1 | SEQ=16
Paquete 17 enviado: 'i' | IP ID=17 | ICMP ID=1 | SEQ=17
Paquete 18 enviado: 'h' | IP ID=18 | ICMP ID=1 | SEQ=18
Paquete 19 enviado: ' ' | IP ID=19 | ICMP ID=1 | SEQ=19
Paquete 20 enviado: 'I' | IP ID=20 | ICMP ID=1 | SEQ=20
Paquete 21 enviado: 'y' | IP ID=21 | ICMP ID=1 | SEQ=21
Paquete 22 enviado: 'h' | IP ID=22 | ICMP ID=1 | SEQ=22
Paquete 23 enviado: 'c' | IP ID=23 | ICMP ID=1 | SEQ=23
Paquete 24 enviado: 'v' | IP ID=24 | ICMP ID=1 | SEQ=24
```

Figura 6: Captura de la terminal de Linux el cual muestra, que se solicita el texto ya cifrado de la actividad número 1, también solicita la dirección IP de destino, en este caso corresponde a 8.8.8.8 (Google), y posterior a ello un listado de cada uno de los paquetes enumerados e indicando el carácter cifrado en aquel paquete.

Mientras se ejecuta el programa en Python de esta segunda actividad se puede observar en Wireshark que se envía el paquete de manera exitosa.

(icmp) && (ip.dst == 8.8.8.8)						
No.	Time	Source	Destination	Protocol	Length	Info
242	79.900898323	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
244	79.940740904	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
246	79.966190518	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
248	79.982890546	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
250	80.021389980	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
252	80.082675449	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request
254	80.103841258	10.0.2.15	8.8.8.8	ICMP	100	Echo (ping) request

▶ Frame 254: 100 bytes on wire (800 bits), 100 bytes captured (800 bits) on interface ▶ Linux cooked capture v1 ▶ Internet Protocol Version 4, Src: 10.0.2.15, Dst: 8.8.8.8 ▼ Internet Control Message Protocol	Type: 8 (Echo (ping) request) Code: 0 Checksum: 0x8ad3 [correct] [Checksum Status: Good] Identifier (BE): 1 (0x0001) Identifier (LE): 256 (0x0100) Sequence Number (BE): 24 (0x0018) Sequence Number (LE): 6144 (0x1800) [Response frame: 255]
▼ Data (56 bytes)	Data: 760102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f20212223... [Length: 56]

0000	00 04 00 01 00 06 08 00	27 03 2d c8 00 00 08 00	 !.....
0010	45 00 00 54 00 18 00 00	40 01 5e 73 0a 00 02 0f	E..T...@.As...
0020	08 08 08 08 08 08 8a d3	00 01 00 18 76 01 02 03	v...
0030	04 05 06 07 08 09 0a 0b	0c 0d 0e 0f 10 11 12 13	
0040	14 15 16 17 18 19 1a 1b	1c 1d 1e 1f 20 21 22 23	!"#
0050	24 25 26 27 28 29 2a 2b	2c 2d 2e 2f 30 31 32 33	\$%&'()*+,-./0123
0060	34 35 36 37		4567

Figura 7: Captura en Wireshark, filtrando por paquetes de tipo ICMP y por su dirección IP, mostrándose el tráfico ICMP generado mediante un ping posterior a la ejecución del programa en Python 3 (paquetes generados). En la captura se muestra como ejemplo uno de los paquetes transmitidos, cuyo campo de datos contiene el carácter ‘v’. Evidenciando que los paquetes se enviaron con éxito y con su finalización correspondiente al paquete normal, el cual es: !"\$% `()*+,-./01234567

En la Figura 7 se observa el tráfico ICMP generado durante la ejecución del programa desarrollado en Python. Al aplicar el filtro correspondiente en Wireshark, es posible identificar múltiples paquetes ICMP de tipo Echo Request, utilizados para transmitir individualmente los caracteres del mensaje cifrado.

La transmisión cumple con la condición de enviar un carácter por paquete, por lo que el mensaje completo queda distribuido entre múltiples solicitudes ICMP y puede ser reconstruido posteriormente respetando el orden de los paquetes.

Al comparar este tráfico con el generado mediante un ping convencional, se observa que ambos utilizan el protocolo ICMP y corresponden a solicitudes Echo Request (Type 8, Code 0). Sin embargo, también existen diferencias en campos como el *Identifier*, el *Sequence Number* y la estructura del campo *Data*.

En el tráfico generado por el programa, el payload incorpora información correspondiente al mensaje cifrado, mientras que el comando `ping` utiliza una estructura de datos propia que incluye información temporal y una secuencia característica de bytes. Por esta razón, aunque el programa logró transmitir correctamente la información mediante paquetes ICMP, no replica completamente todas las características del tráfico generado por un ping convencional.

Las capturas realizadas en Wireshark permiten comprobar la presencia de los caracteres transmitidos dentro del payload y verificar que el mensaje cifrado fue distribuido correctamente entre múltiples paquetes ICMP. De esta forma, se cumplió el objetivo de la actividad y se generó el tráfico necesario para la posterior recuperación y reconstrucción del mensaje en la siguiente actividad.

3.3. Actividad 3

En esta tercera y última actividad del laboratorio, se pide nuevamente a la herramienta de IA ChatGPT, generar un programa en Python 3, que permita obtener el mensaje transmitido en la segunda actividad.

Como no se sabe el desplazamiento utilizado en el texto cifrado, se le pide a este programa que pueda crear todas las combinaciones posibles, ya que estamos trabajando con el alfabeto en inglés, son 26 posibilidades de mensaje. Estas deben ser mostradas por pantalla e indicando con color verde la opción más probable de corresponder al mensaje original en texto claro.

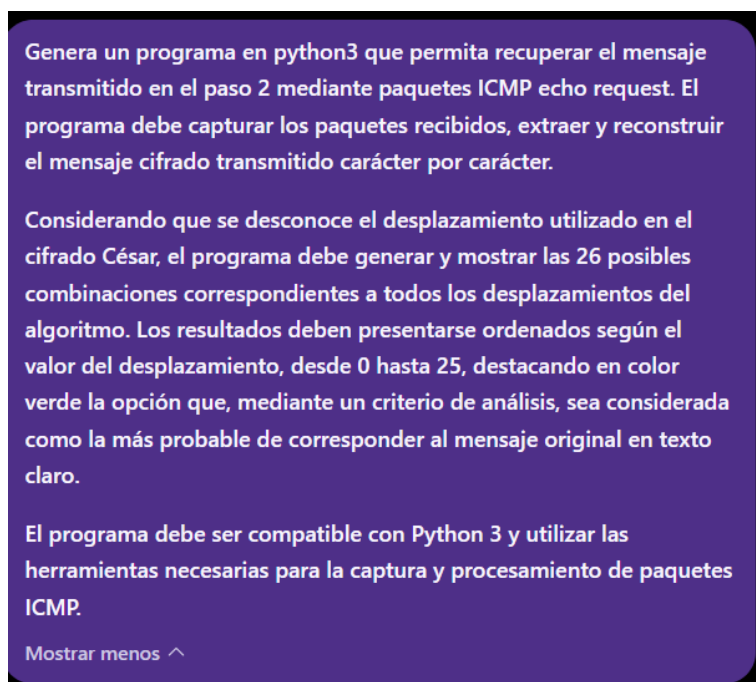
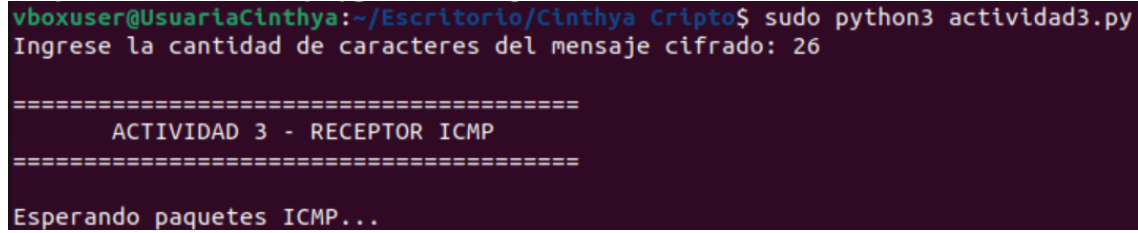


Figura 8: Prompt utilizado en la herramienta de IA (ChatGPT) para la generación del código en Python 3, para la actividad número 3, solicitando mostrar en pantalla todas las posibles combinaciones.



```
vboxuser@UsuarialaCinthya:~/Escritorio/Cinthya Cripto$ sudo python3 actividad3.py
Ingrese la cantidad de caracteres del mensaje cifrado: 26

=====
          ACTIVIDAD 3 - RECEPTOR ICMP
=====

Esperando paquetes ICMP...
```

Figura 9: En esta captura de la terminal se logra apreciar la ejecución del tercer programa de Python 3 llamado `actividad3.py`. Este programa solicita la cantidad de caracteres del paquete cifrado en este caso corresponde a 26. Luego, solicita que se ejecute nuevamente el programa realizado en la actividad 2, para que se envíen los mensajes que se desean descifrar.

Una vez que se realiza nuevamente la ejecución del programa en la actividad numero dos, la terminal muestra la ejecución del programa de esta tercera actividad el cual evidencia lo siguiente:

```
Esperando paquetes ICMP...
Carácter recibido: J
Mensaje cifrado actual: J
Carácter recibido: p
Mensaje cifrado actual: Jp
Carácter recibido: u
Mensaje cifrado actual: Jpu
Carácter recibido: a
Mensaje cifrado actual: Jpua
Carácter recibido: o
Mensaje cifrado actual: Jpuao
Carácter recibido: f
Mensaje cifrado actual: Jpuaof
Carácter recibido: h
Mensaje cifrado actual: Jpuaofh
Carácter recibido:
Mensaje cifrado actual: Jpuaofh
Carácter recibido: M
Mensaje cifrado actual: Jpuaofh M
Carácter recibido: b
Mensaje cifrado actual: Jpuaofh Mb
Carácter recibido: l
Mensaje cifrado actual: Jpuaofh Mbl
Carácter recibido: u
Mensaje cifrado actual: Jpuaofh Mblu
Carácter recibido: a
Mensaje cifrado actual: Jpuaofh Mblua
Carácter recibido: l
Mensaje cifrado actual: Jpuaofh Mblual
Carácter recibido: h
Mensaje cifrado actual: Jpuaofh Mblualh
Carácter recibido: s
Mensaje cifrado actual: Jpuaofh Mblualhs
Carácter recibido: i
Mensaje cifrado actual: Jpuaofh Mblualhsi
Carácter recibido: h
Mensaje cifrado actual: Jpuaofh Mblualhsih
Carácter recibido:
Mensaje cifrado actual: Jpuaofh Mblualhsih
```

Figura 10: Captura parte 1 de la terminal de Linux ejecutando el tercer programa en Python 3, el cual muestra los paquetes recibidos y el carácter oculto del texto cifrado, de cada paquete.

```
Carácter recibido:  
Mensaje cifrado actual: Jpuaofh Mblualhsih  
Carácter recibido: I  
Mensaje cifrado actual: Jpuaofh Mblualhsih I  
Carácter recibido: y  
Mensaje cifrado actual: Jpuaofh Mblualhsih Iy  
Carácter recibido: h  
Mensaje cifrado actual: Jpuaofh Mblualhsih Iyh  
Carácter recibido: c  
Mensaje cifrado actual: Jpuaofh Mblualhsih Iyhc  
Carácter recibido: v  
Mensaje cifrado actual: Jpuaofh Mblualhsih Iyhcv  
^C  
=====
```

MENSAJE CIFRADO COMPLETO

```
=====
```

Jpuaofh Mblualhsih Iyhcv

Figura 11: Captura parte 2 de la terminal de Linux ejecutando el tercer programa en Python 3, el cual muestra los paquetes recibidos en su totalidad. Finalmente en la parte inferior se observa el mensaje recuperado, juntando todos los caracteres.

```

=====
26 POSIBILIDADES CIFRADO CÉSAR
=====
Desplazamiento 0: Jpuaofh Mblualhsih Iyhcv
Desplazamiento 1: Iotzneg Laktzkgrhg Hxgbu
Desplazamiento 2: Hnsymdf Kzjsyjfqqf Gwfat
Desplazamiento 3: Gmrxlce Jyirxiepe Fvezs
Desplazamiento 4: Flqwkbd Ixhqwhdoed Eudyr
Desplazamiento 5: Ekpvjac Hwgpvgcndc Dtcxq
Desplazamiento 6: Djouizb Gvfoufbmcb Csbwp
Desplazamiento 7: Cinthya Fuentealba Bravo
Desplazamiento 8: Bhmsgxz Etdmsdzkaz Aqzun
Desplazamiento 9: Aglrfwy Dsclrcyzy Zpytm
Desplazamiento 10: Zfkqevx Crbkqbxlyx Yoxsl
Desplazamiento 11: Yejpduw Bqajpawhwx Xnwrk
Desplazamiento 12: Xdioctv Apziozvgwv Wmvqj
Desplazamiento 13: Wchnbsu Zoyhnyufvu Vlupi
Desplazamiento 14: Vbgmart Ynxgmteut Uktoh
Desplazamiento 15: Uaflzqs Xmwflwsdts Tjsng
Desplazamiento 16: Tzekypr Wlvekvrcsr Sirmf
Desplazamiento 17: Sydjxoq Vkudjuqbrq Rhqle
Desplazamiento 18: Rxcwnp Ujtcitpaqp Qgpkd
Desplazamiento 19: Qwbhvmo Tisbhszopo Pfojc
Desplazamiento 20: Pvaguln Shragrnyon Oenib
Desplazamiento 21: Ouzftkm Rgqzfqmxnm Ndmha
Desplazamiento 22: Ntyesjl Qfpyeplwml Mclgz
Desplazamiento 23: Msxdrik Peoxdokvlk Lbkfy
Desplazamiento 24: Lrwcqhj Odnwcnjukj Kajex
Desplazamiento 25: Kqvbpgi Ncmvbmijti Jzidw

```

Figura 12: Captura de la terminal de Linux mostrando en pantalla todas las combinaciones posibles existentes en total 26, y mostrando en color verde, la respuesta correcta con el texto descifrado y legible. en este caso corresponde a un desplazamiento en el algoritmo César de 7.

Finalmente en la Figura 12 se puede apreciar que la opción identificada con color verde como la más probable corresponde correctamente al mensaje original: “Cinthya Fuentealba Bravo”. Las posibilidades se encuentran en orden ascendiente desde el desplazamiento 0 hasta el desplazamiento 25, abarcando así en su totalidad todas las combinaciones posibles.

4. Conclusiones y comentarios

El desarrollo de este laboratorio permitió integrar conceptos relacionados con la criptografía, protocolos de red y análisis de tráfico.

En primer lugar, se implementó un programa que ejecutara el algoritmo de cifrado César, y a su vez observar y aprender el método de cifrado basado en el desplazamiento de caracteres. Posteriormente, el mensaje cifrado fue transmitido utilizando paquetes ICMP, fragmentando la información mediante el envío de un carácter por paquete. Finalmente, se demostró que el mensaje podía ser recuperado mediante el análisis del tráfico y la generación de las diferentes

posibilidades de descifrado.

La realización de este laboratorio permitió entender y aprender que el algoritmo César posee importantes limitaciones de seguridad debido a la reducida cantidad de combinaciones posibles.

Por otra parte, el laboratorio permitió experimentar con el uso de IA como herramienta de apoyo para el desarrollo de software. Si bien ChatGPT facilitó la generación inicial de código y la comprensión de algunos conceptos, fue necesario validar los resultados obtenidos mediante pruebas y realizar modificaciones para cumplir completamente con los requerimientos.

En conclusión, la experiencia permitió comprender que la IA puede ser una herramienta útil para apoyar el desarrollo de programas, pero sus resultados deben ser revisados y validados por el usuario. Asimismo, el laboratorio permitió relacionar conceptos de criptografía con la transmisión y análisis de información en redes.

5. Uso de IA

Para el desarrollo de las actividades de este laboratorio se utilizó ChatGPT como herramienta de Inteligencia Artificial Generativa.

La elección de esta herramienta se debe principalmente a su capacidad para asistir en la generación de código en diferentes lenguajes de programación, explicar el funcionamiento de los programas generados y permitir realizar modificaciones a partir de instrucciones entregadas en lenguaje natural.

Otra característica relevante fue la posibilidad de realizar un proceso iterativo. Esto permitió solicitar modificaciones al código cuando el resultado inicial no cumplía completamente con los requerimientos establecidos.

Sin embargo, la utilización de una herramienta de IA no elimina la necesidad de revisar y validar el código generado. Durante el desarrollo fue necesario ejecutar los programas, analizar sus resultados y realizar ajustes en los prompts para obtener un resultado que cumpliera con las condiciones específicas del laboratorio.

A continuación se mencionan y describen cuatro problemáticas identificadas durante el desarrollo del laboratorio relacionadas con el uso de IA:

■ **Problema 1:**

El primer resultado no siempre cumplía todos los requisitos, ya que en algunos casos el código generado inicialmente cumplía la idea general solicitada, pero no consideraba todos los detalles específicos establecidos en el laboratorio. Por esta razón, fue necesario mejorar y especificar los prompts entregados a la herramienta.

- **Problema 2:**

El código generado por la herramienta no podía considerarse correcto únicamente por haber sido generado. Fue necesario ejecutar los programas en Linux y comprobar que los resultados correspondieran con lo solicitado en el laboratorio.

- **Problema 3:**

Durante la transmisión por medio de paquetes ICMP fue necesario especificar detalladamente la forma en que cada carácter debía ser enviado. Para obtener el comportamiento esperado, fue necesario indicar explícitamente que los caracteres debían conservar su orden y que cada uno debía transmitirse de forma individual.

- **Problema 4:**

Se observó que mientras más general era la instrucción entregada a la IA, mayor era la posibilidad de obtener un resultado incompleto o diferente a lo esperado. Por esta razón, fue necesario entregar instrucciones más detalladas, indicando las entradas, salidas y condiciones específicas que debía cumplir cada programa.

En el siguiente repositorio de GitHub se encuentra la información completa de este laboratorio: Repositorio del laboratorio 1 en GitHub.